

Evaluating and Refining AI Outputs

Nathan J. McNeese

Associate Vice President for Technology & Innovation

McQueen Quattlebaum Endowed Professor

Founding Director, Clemson Center for Human-AI Interaction, Collaboration,
& Teaming

Founding Director, TRACE Research Group



Evaluation determines whether an AI output is usable

AI can produce an answer, code, or analysis quickly. Evaluation determines whether the output is correct, supported, and appropriate for the task.

The AI output

- Answer or explanation
- Code or calculation
- Analysis or recommendation

The evaluation questions

- Did it answer the right task?
- Is it correct and supported?
- Can the work be inspected?
- Are limitations visible?

Evaluation converts an AI response into an evidence-based judgment.

Good evaluation moves from the surface to the evidence

Start with what the AI produced. Then test the evidence and work behind it.

Surface check

- Relevant to the task
- Clear and complete
- Appropriate for the audience

Evidence check

- Claims are correct and supported
- Work can be inspected or reproduced
- Assumptions and uncertainty are visible

Define what good looks like before judging the output

Write down what a good response must accomplish for this task.

Your checks should cover:

- Correct objective and audience
- Evidence supports the conclusions
- The work can be inspected or reproduced
- Uncertainty and limitations are visible

Three questions

1
Right task?

2
Verifiable work?

3
Appropriate to use?

The level of review should match the risk

Consequential, hidden, or hard-to-reverse errors require stronger evidence.

Ask:

- What decision could this output influence?
- How costly would an error be?
- How easily could I detect a problem?
- Can I verify the output independently?
- Can I reverse the decision if it is wrong?

Evaluation requires an evidence trail

You cannot verify an AI output if its inputs, changes, and prior versions disappear.

Keep:

- The prompt, inputs, and constraints
- The complete AI response
- Sources, calculations, code, or changes
- Tests, warnings, and evaluation results
- A known-good version for comparison

Step 1: Did AI answer the actual task?

A coherent response can still solve a different—or easier—problem.

Check:

- Correct objective
- Correct audience or user
- Required constraints were followed
- Requested scope and format were preserved
- No unapproved change in the task

Step 2: Is each claim correct and supported?

Evaluate both the accuracy of the claim and the evidence used to justify it.

Check correctness

- Facts are accurate
- Calculations match the inputs
- The response is internally consistent

Check support

- Claims trace to a source, result, or test
- Observation is separated from inference
- Limitations are stated

Step 3: Separate what is known from what is inferred

AI often blends evidence, interpretation, and recommendation. Pull them apart.

Known

The dataset contains 12% missing values.

Inferred

The missingness may bias the result.

Next step

Inspect the missingness pattern before choosing a method.

Step 4: Inspect the work behind the answer

A conclusion is easier to evaluate when the path to it is visible.

Ask AI to show:

- The sources or inputs it used
- Calculations, logic, or code
- Changes made to data or files
- Assumptions and decisions
- Tests or checks already performed

Step 5: Test the output independently

Do not rely on the AI's explanation of its own performance.

For claims or analysis

- Check key claims against the source
- Recompute a sample calculation
- Test edge cases or counterexamples

For code or models

- Run the work yourself
- Inspect errors and warnings
- Compare with a baseline
- Reproduce with the same inputs

Step 6: End evaluation with a specific finding

Revision begins only after you can name what passed, what failed, and why.

Document:

- What passed
- What failed
- The evidence for that finding
- The severity or impact
- What must remain unchanged

Name the failed component before revising anything.

Revision starts with diagnosis—not a new answer

Different failures require different revisions. Identify the source first.

Task failure

- Wrong objective
- Missing constraint

Evidence failure

- Unsupported claim
- Incorrect data or calculation

Output failure

- Unclear explanation
- Wrong format or tone

Step 1: Identify the smallest failing component

Fix the cause of the failure—not every part of the response.

The failing component may be:

- An instruction or scope decision
- A factual claim
- Data, a calculation, or code
- Reasoning or explanation
- Format, tone, or level of detail

Step 2: Decide what to preserve and what to change

Revision should protect the parts that already passed evaluation.

Preserve

- Verified claims
- Correct calculations or code
- Useful structure
- Required constraints

Change

- The failed claim or step
- Unsupported assumptions
- Faulty logic or code
- Unclear wording

Step 3: Give AI evidence-based feedback

Tell AI exactly what failed and the evidence that revealed the problem.

The problem

- Identify the exact claim, step, or file
- Describe the expected result
- Explain the consequence

The evidence

- Source or reference
- Test result or error message
- Correct calculation or example
- Constraint that was missed

Step 4: Bound the revision

A bounded request makes the change easier to inspect and compare.

Define the boundary

- Change only the failed component
- Preserve accepted content
- Keep the same inputs and constraints

Define the deliverable

- Return the revised section or code
- Explain what changed
- Show updated tests or checks

Step 5: Request one focused revision

A strong revision request combines the failure, the evidence, and the required change.

Failure

- What failed
- Where it appears

Evidence

- How you know
- Expected result

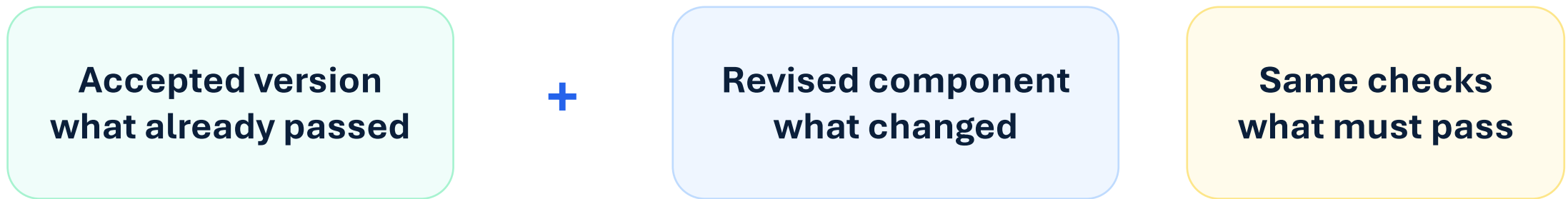
Revision

- What must change
- What must remain

Change one component, then evaluate it again.

Step 6: Compare the revision with the accepted version

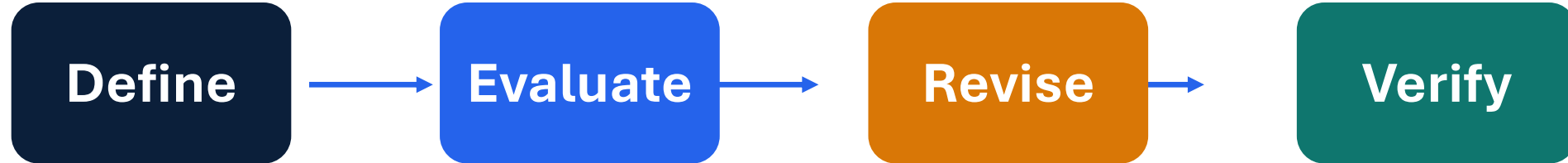
Re-evaluate the change and check that previously accepted work still passes.



- Did the revision fix the failure?
- Did it introduce a new problem?
- Keep the known-good version until the revision passes.

Revision is complete only after re-evaluation

Evaluate first. If a component fails, revise it deliberately and run the same checks again.



Clear criteria • evidence • focused revision • verified result